

Praktyczne spojrzenie na konkursy programistyczne

**Czyli o skutecznej nauce algorytmiki
na portalu leetcode.com**

Opowiada: Adam Graliński

Agenda

Czyli co dziś w menu:

- 3. Wstęp i opis zagadnienia
- 6. Opis platformy leetcode.com
- 9. Polecane listy zadań
- 11. Subiektywnie: co działa, a co nie działa. Pułapki i pochlaniacze czasu.
- 12. Śledzenie postępów
- 14. Rola edytora tekstu
- 16. Integracja z gitem
- 19. Co dalej?

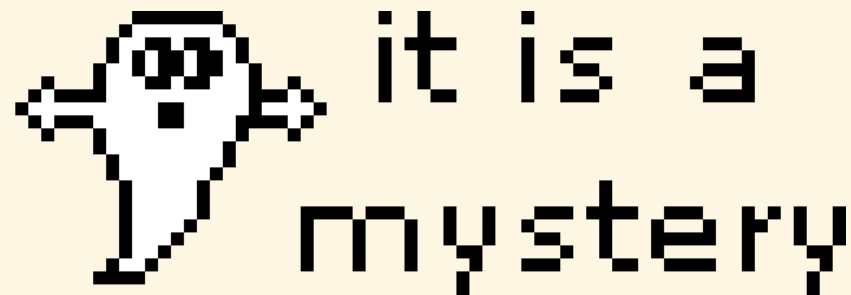
Konkursy programistyczne

(ang. *competitive programming*)

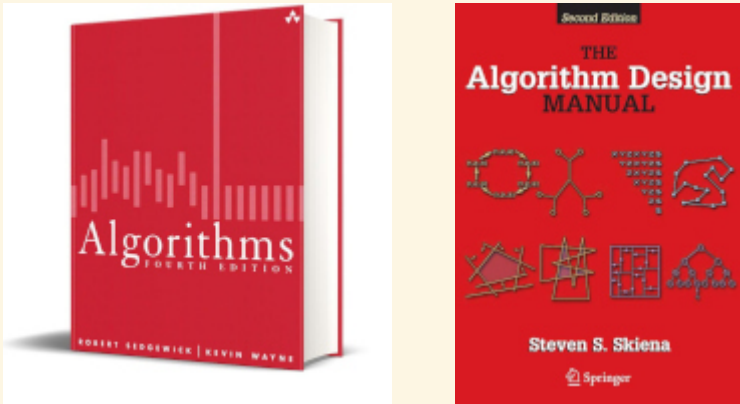
- https://en.wikipedia.org/wiki/Competitive_programming

Jak skutecznie zaliczyć SW Test?

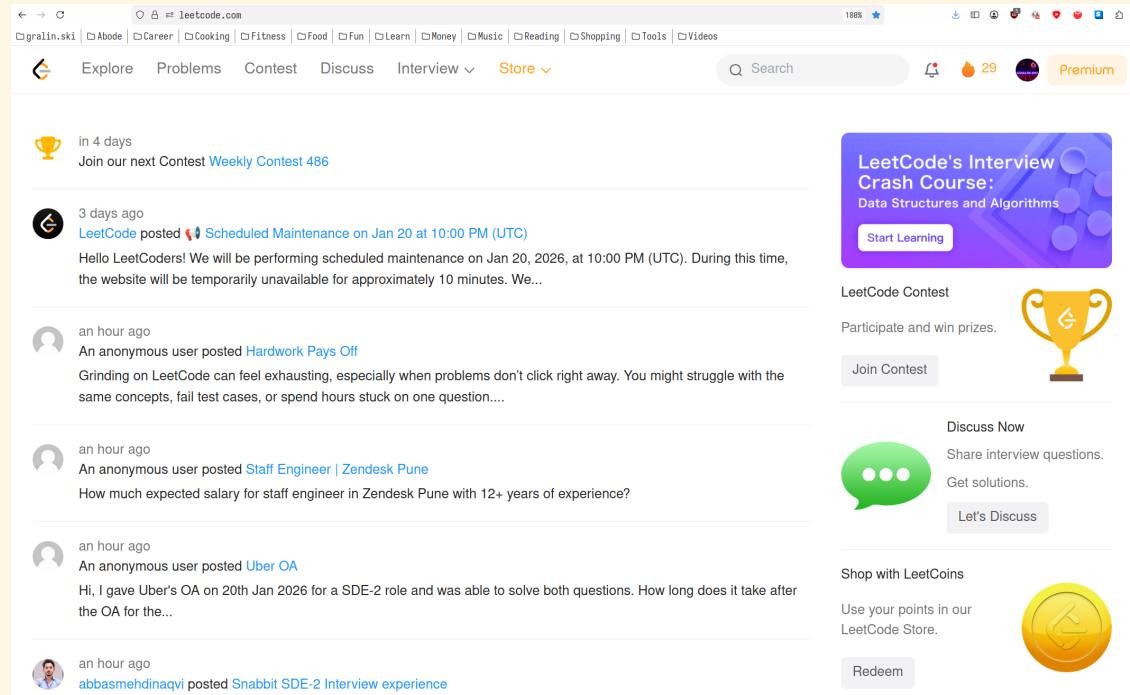
(jak skutecznie wygrać konkurs programistyczny?)



Jak skutecznie zaliczyć SW Test?

Teoria	Praktyka
 The image shows two books. On the left is 'Algorithms' by Robert Sedgwick and Kevin Wayne, Fourth Edition, published by Springer. It has a red cover with a white bar chart graphic. On the right is 'The Algorithm Design Manual' by Steven S. Skiena, Second Edition, published by Springer. It has a red cover with various algorithmic diagrams and the title in white and yellow text.	• Konkursy programistyczne ◦ <u>Leetcode</u> ◦ <u>Advent of Code</u> ◦ <u>Project Euler</u> ◦ <u>OverTheWire</u> • Własne projekty ◦ <u>Udemy</u> (np. SamsungU) ◦ <u>Coursera</u> ◦ <u>Khan Academy?</u>

Opis platformy leetcode.com



Opis platformy leetcode.com

The screenshot displays the LeetCode website interface. On the left, the 'Daily Question' section lists 15 problems with their difficulty levels (Easy, Medium, Hard). The problem '3314. Construct the Minimum Bitwise Array I' is highlighted at the bottom. On the right, a code editor shows a Go solution for this problem. The code includes comments about runtime complexity and auxiliary space, and a function `minBitwiseArray` that uses a helper function `getFirstSetBitOfLastGroupOfSetBits`.

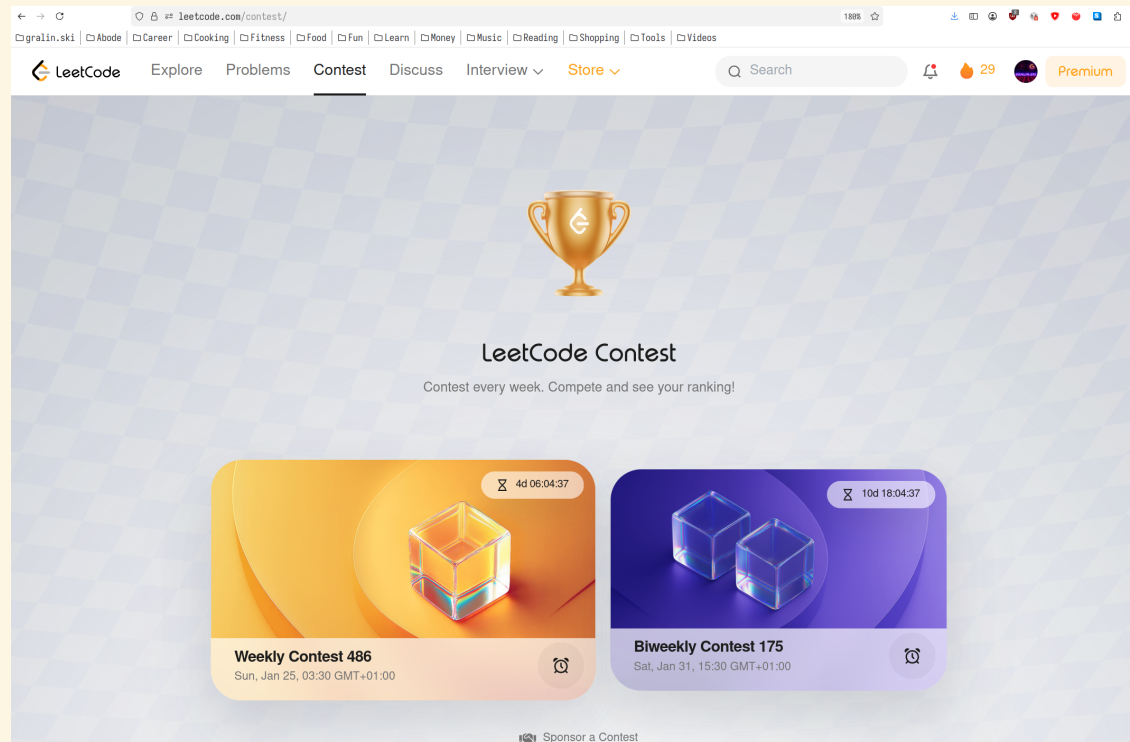
Daily Question

- ✓ 1161. Maximum Level Sum of a Binary Tree (Med.)
- ✓ 1339. Maximum Product of Splitted Binary Tree (Med.)
- ✓ 1458. Max Dot Product of Two Subsequences (Hard)
- ✓ 865. Smallest Subtree with all the Deepest Nodes (Med.)
- ✓ 712. Minimum ASCII Delete Sum for Two Strings (Med.)
- ✓ 85. Maximal Rectangle (Hard)
- ✓ 1266. Minimum Time Visiting All Points (Easy)
- ✓ 3453. Separate Squares I (Med.)
- ✓ 3454. Separate Squares II (Hard)
- ✓ 2943. Maximize Area of Square Hole in Grid (Med.)
- ✓ 2975. Maximum Square Area by Removing Fences From a Field (Med.)
- ✓ 3047. Find the Largest Area of Square Inside Two Rectangles (Med.)
- ✓ 1895. Largest Magic Square (Med.)
- ✓ 1292. Maximum Side Length of a Square with Sum Less than or Equal to... (Med.)
- ✓ 3314. Construct the Minimum Bitwise Array I (Easy)

```
// Runtime complexity: O(N * sum(log(nums[i])))  
// - hard to estimate, but higher than N, and surely lower than N^2  
// - probably close to O(N*logN) or maybe to O(N).  
// Auxiliary space: O(N) (for holding the answer; otherwise O(1)).  
// Subjective level: easy+. Requires bit shifting, but nothing fanc  
// Solved on: 2026-01-20  
func minBitwiseArray(nums []int) []int {  
    N := len(nums)  
    ans := make([]int, N)  
    getFirstSetBitOfLastGroupOfSetBits := func(num int) int {  

```

Opis platformy leetcode.com



Polecane listy zadań

<u>Leetcode 75</u>	<u>Blind 75</u> (alias <u>Neetcode 75</u>)	Extended 150 (alias <u>Neetcode 150</u>)
+ Przekrój różnych tematów + Dostępne bez konta premium + Rozwiązania ogólnodostępne	+ (Dokładny) przekrój różnych tematów - 6/75 zadań wymaga premium + Możliwość obejrzenia rozwiązywania na żywo + <u>T0u5nwSA0w0</u> ± (zaledwie 40 godzin)	

Kategoryzacja tematów

- **Backtracking**
- Binary search
- **Dynamic programming**
- Grafy: BFS i DFS
- Grafy: Dijkstra
- Grafy: indegree/outdegree (-)
- Greedy, czyli algorytmy zachłanne
- **Hashing**
- **Heap / Priority Queue**
- Matrix, czyli problemy tablicowe
- **Prefix Sum**
- Sliding Window
- Stack
- String, czyli łańcuchy tekstowe
- Trees (a zwłaszcza Binary Trees)
- **Two Pointers**
- Union Find
- Combinatorics (-)
- Trie (-)

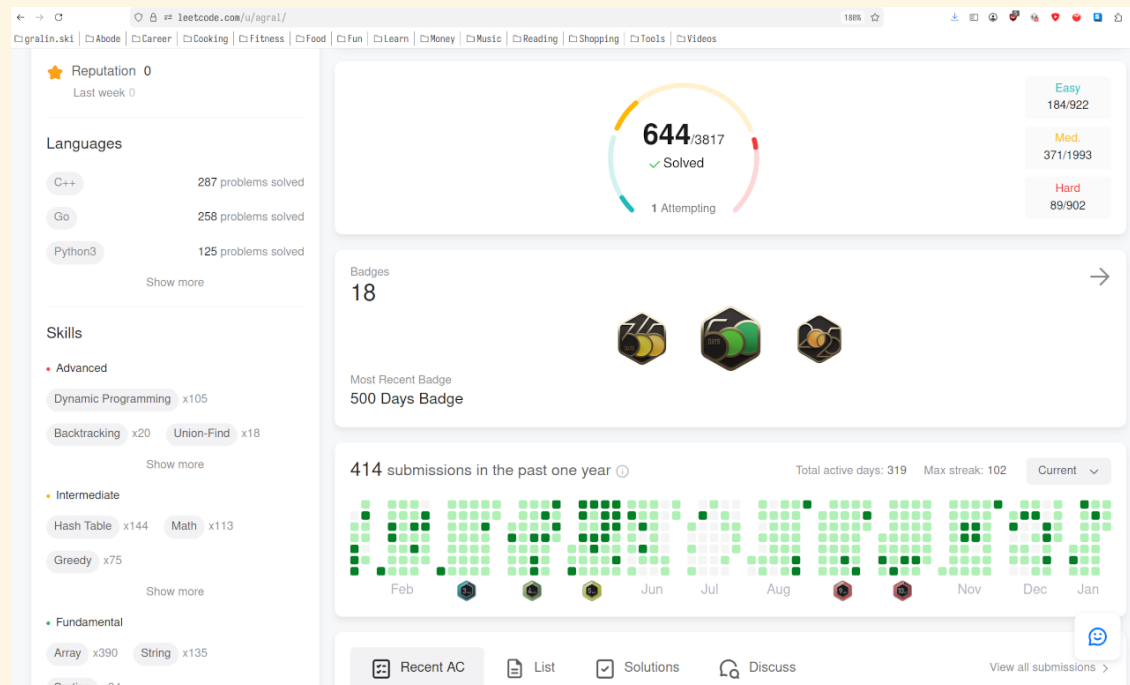
Działa, nie działa...

czyli pułapki i pochtaniacze czasu.

Działa	Nie działa
+ Samodzielne rozwiązywanie zadań + Korzystanie z pomocy: literatura, blogi, Wikipedia itd. + Własnoręczne ocenianie trudności zadań + Wypracowanie własnego systemu	- Bezmyślne przepisywanie rozwiązań z sieci - Usilne, beznadziejne próby samodzielnego rozwiązania gdy nie znamy tematu - Grinding. Odpocznij!

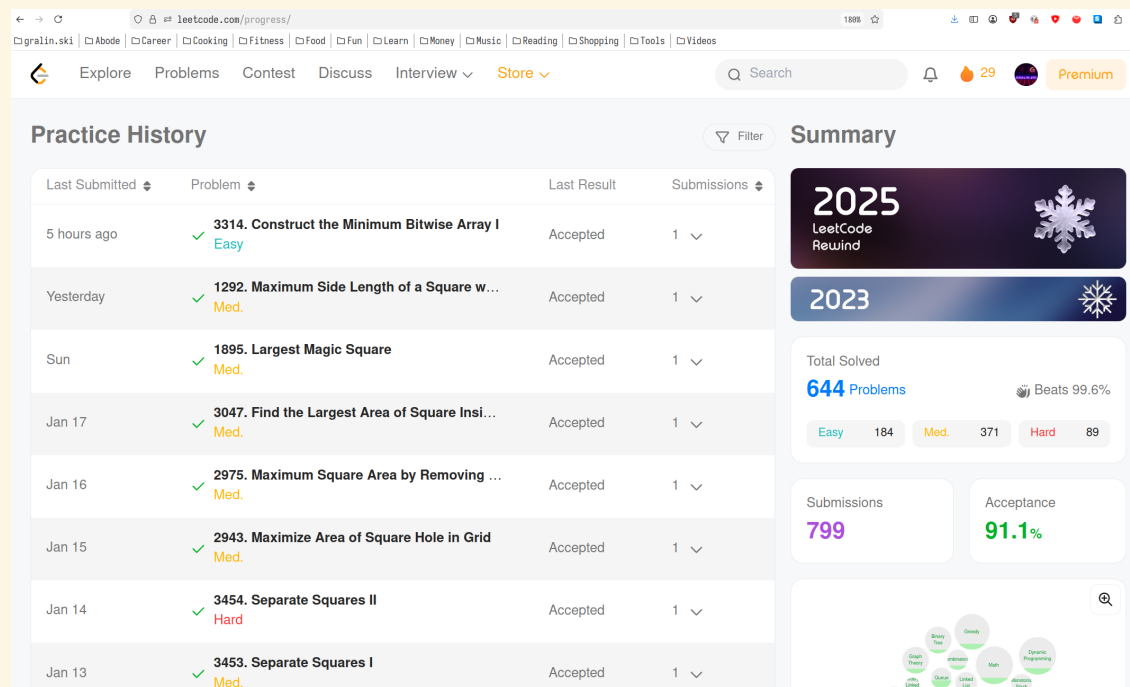
Śledzenie postępów

Banalnie proste. Cały portal jest mocno zgamifikowany.



Śledzenie postępów

Banalnie proste. Cały portal jest mocno zgamifikowany.



Edytor tekstu

- Zadania rozwiązuj we własnym edytorze tekstu
- Optymalnie: tym samym, którego używasz na SW Test
 - pamięć mięśniowa
 - spróbuj maksymalnie wykorzystać możliwości edytora

Edytor tekstu

C++ dotfile on github	Python dotfile on github
<pre>#include <iostream> // Runtime complexity: // Auxiliary space complexity: // Subjective level: // Solved on: 2020-01-21 class Solution { public: int solve(int input) { int ans = {}; // TODO return ans; } }; int main() { struct Testcase { int input; int expected; }; Testcase testcases[] = { {}, }; Solution s; int numGood = 0, numBad = 0; for (Testcase& tc: testcases) { auto actual = s.solve(tc.input); if (actual != tc.expected) { std::cout << "Testcase " << tc.input << " failed. Got: " << actual << ", want: " << tc.expected << "\n"; ++numBad; } else { ++numGood; } } std::cout << (numBad == 0 ? "[OK]" : "[FAIL]") << " " << numGood << "/" << (numBad + numGood) << " testcases passed successfully.\n"; }</pre>	<pre>from typing import List // Runtime complexity: // Auxiliary space complexity: // Subjective level: // Solved on: 2020-01-21 class Solution: def solve(self, input: List[int]) -> int: ans = -42 # TODO return ans</pre>

Integracja z gitem

- Zachowuj stworzone przez siebie rozwiązania zadań!
- Użyj do tego osobnego repozytorium git
 - Uniezależnienie od konkretnej platformy
 - Możliwość trzymania rozwiązań zadań z różnych platform w jednym miejscu

2022-08-15 | [b65cf7a](#), [LC0013](#), "Roman To Integer"

Integracja z gitem

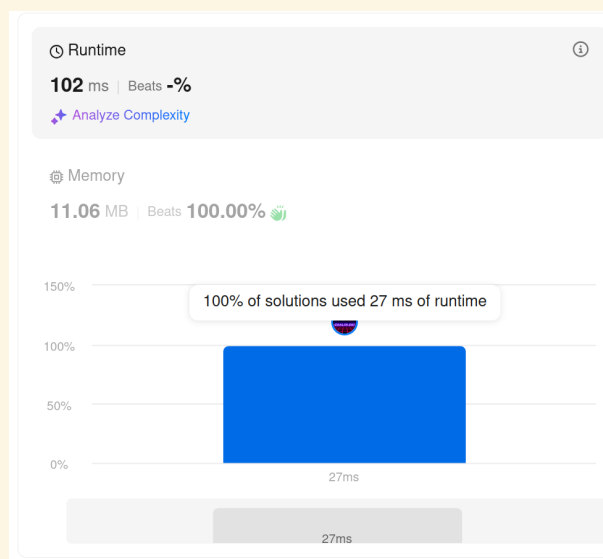
The screenshot shows a GitHub repository page for 'CompetitiveProgramming'. The repository is public and has 1 branch (master) and 0 tags. The commit history shows a recent commit by 'agral' titled 'Solve 3314/Construct the Minimum Bitwise Array I' 15 hours ago. The repository contains several folders and files, including 'AdventOfCode', 'AoC2017', 'AoC2019', 'HackerRank', 'LeetCode', 'SPOJ', '.gitignore', 'README.md', and 'go.mod'.

File/Folder	Description	Time
AdventOfCode	Solve AoC2025 / Day04	last month
AoC2017	Add a screenshot of main calendar of AoC2017	9 years ago
AoC2019	Add Intcode interpreter for AoC2019's challenges	7 years ago
HackerRank	Add HR's Algo/Impl/AlmostSorted solution in Java 8	6 years ago
LeetCode	Solve 3314/Construct the Minimum Bitwise Array I	15 hours ago
SPOJ	Add solutions to Sphere Online Judge (2011-2014)	2 years ago
.gitignore	Ignore __pycache__ contents in python solutions	10 months ago
README.md	Update a top-level README.md file	9 years ago
go.mod	Declare a go module for the entire repo	2 years ago

Keep calm & carry on...

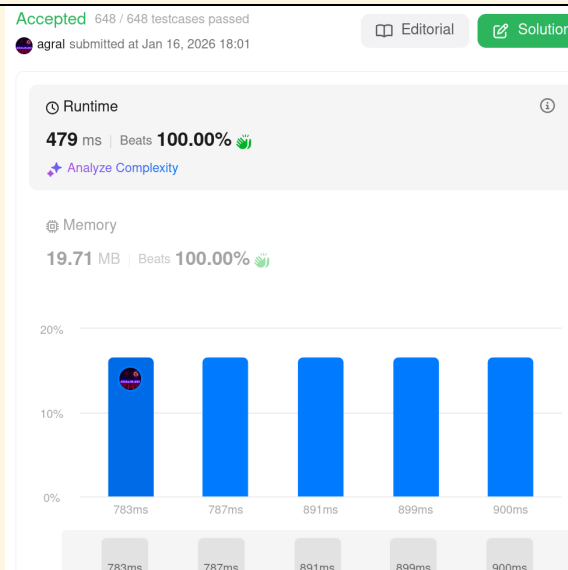
Czasem przegrywasz...

LC1970



Czasem wygrywasz...

LC2975



Słowa końcowe

- Wypróbuj to przez jeden miesiąc!
- Kod który piszemy na SW teście nie jest kodem produkcyjnym!
 - Bez "zbędnych" asercji, które zjadają cenne mikrosekundy ([LC3047](#) | [github](#))
- Co gdy już zdasz...?
 - Przykład: integracja LSP, DAP etc. z edytorem

Dzięki!

Źródła mediów użytych w prezentacji

- 04, [it is a mystery](#)
- 05, ["Algorytmy" Sedgewicka & Wayne'a](#)
- 05, ["The Algorithm Design Manual" Skieny](#)